

ComplexGitSync 4.2.1– Getting Started

Nicolas Flipó

October 6, 2026

Contents

1	Getting Started	1
2	What ComplexGitSync is for	1
3	The project lifecycle	2
4	Public and private repositories	3
5	Memory and persistence	3
6	Installation	4
7	Standalone or nested	4
8	A first project	5
9	Day-to-day work	5
10	Getting help	6
11	Next steps	6

1 Getting Started

This chapter takes a new user from installation to a first persisted State of a project. It follows the structure of the project’s `README.md`, with the detail needed to act on each part. Every command and option is described in full in the User Guide.

2 What ComplexGitSync is for

Scientific projects are becoming more complex: beyond source code, they rely on documentation, tutorials, data, continuous integration and deployment, and the persistence of their results — each often held in a Git repository of its own. **ComplexGitSync** manages such a project as a distributed, git-native operating space, in the form of a **GitTree**: a tree of nested Git repositories that is synchronised, versioned and restored as one unit.

A project can couple **public and private repositories**, so that intellectual property is preserved even when the project is developed collaboratively. Private repositories can also carry the configuration of **agentic assistance** for different parts of the project. Every state the project goes through is recorded, which gives the project **persistence and reproducibility**: any recorded state can be rebuilt, exactly, on another machine.

3 The project lifecycle

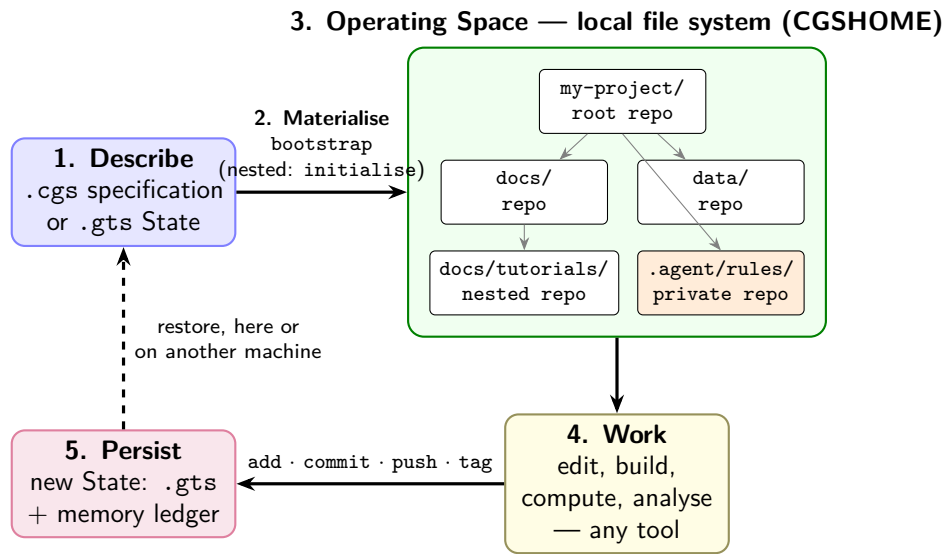


Figure 1: The lifecycle of a project: a description is materialised as a local file system, the Operating Space; the project is worked on with any tool; its new state is persisted, and can be restored from there.

Figure 1 shows the five steps every project goes through. Each is described below with the vocabulary the rest of this manual uses.

1. Describe. The project’s topology is written once in a `.cgs` specification, a short TOML file: which repositories the project is made of, where each one sits in the tree, and which are private. A `.cgs` names branches, so it always gives the latest work. A `.gts` snapshot recorded earlier can be used in its place: it names exact commits, so it gives the project exactly as it was.

```

project = "my-project"
repos = [
  "github:me/my-project",      # the root repository
  "github:me/my-project-docs", # a project repository
  { repository = "github:me/.myRules",
    private = true, writable = true }, # a private one
]
```

Each identifier has the form `provider:owner/name`; the providers are `github`, `gitlab`, `codeberg` and `custom`. Branch, protocol and path all have defaults, and the file `examples/template.cgs` is a starting point. Before anything is cloned, `validate` checks a `.cgs` and `view-tree` draws the tree it describes:

```

$ pixi run cgitssync validate my-project.cgs
$ pixi run cgitssync view-tree my-project.cgs
```

2. Materialise. `bootstrap`, the generic command for any kind of project, clones every repository the description names and places it at its path in the tree. `initialise` is its nested counterpart, run from inside the project (Section 7).

3. The Operating Space. The result is an ordinary directory tree on the local disk, called the project’s `CGSHOME`. Some of its sub-directories are Git repositories of their own, public or

private, nested at the paths the description gave them. ComplexGitSync keeps its own records in `CGSHOME/.cgitsync/`: the States, the ledger, and a log of every command it ran.

4. Work. The project files are used with any tool, exactly as in any other directory: editors, compilers, notebooks, simulation codes, plain `git` inside one repository. ComplexGitSync does not intervene.

5. Persist. `cgitsync` runs each Git operation across the whole tree, in the right order, and checks every repository before changing any of them. Each command that changes the tree records the resulting **State**: a `.gts` snapshot holding the exact commit of every repository, entered in the project's **memory**, a hash-chained ledger that can be verified. A State can be given to `bootstrap` (or, in the nested mode, to `initialise`) to restore it — which closes the cycle.

4 Public and private repositories

A GitTree holds two kinds of repositories.

Project repositories are the work itself: code, documents, public data. They all follow the project's branch: `cgitsync checkout feature` moves every one of them to `feature`.

Private repositories are how the project is run: pipelines, agent instructions, internal rules, private data. They are declared with `private = true`, are often shared between several projects, and keep their own branch rather than following the project's.

A private repository is read-only unless its entry adds `writable = true`. A writable one is written to only on request: commands act on the project repositories by default, and on the writable private ones with `-private`. ComplexGitSync never writes to a read-only one.

```
$ pixi run cgitsync commit "update the analysis"           # project  
    repositories  
$ pixi run cgitsync commit --private "tighten a rule"      # writable  
    private ones
```

tutorials/04_private_repos.md works through a complete example.

5 Memory and persistence

Every State, and every operation that produced one, is entered in the project's memory, together with the tool versions and the environment it ran in. A project with no private repository keeps its memory on the local disk only. A project with private repositories is expected to keep it in a private repository of its own, so that its history outlives any single machine; `cgitsync` offers to set this up the first time it records something:

```
$ pixi run cgitsync memory setup          # create and adopt the memory  
    repository  
$ pixi run cgitsync memory list           # every recorded State, newest  
    first  
$ pixi run cgitsync memory as-of 2026-09-01 # the project on that date  
$ pixi run cgitsync verify check          # does the recorded history verify  
    ?
```

Once adopted, the memory is sent with every `push`, `tag` and `freeze-release`. tutorials/05_memory.md covers it in full.

6 Installation

Tool	Why it is needed
Git	Clones and synchronises every repository.
Pixi	Installs the locked Python environment and runs <code>cgitsync</code> ; Python itself comes with it.
gh, glab or tea	Optional: creates repositories on GitHub, GitLab or Codeberg (<code>repo create</code> , <code>memory setup</code>).

Git authentication must also work for every remote the project uses.

```
$ git clone https://github.com/flipoyo/ComplexGitSync.git
$ cd ComplexGitSync
$ pixi install
$ pixi run cgitsync --help
```

`cgitsync` is a Pixi task, not a globally installed program: every command is typed as `pixi run cgitsync ...` from inside the `ComplexGitSync` clone.

7 Standalone or nested

The `ComplexGitSync` clone can manage a project from outside it or from inside it. The choice is made once, by the command used to materialise the project.

Standalone (recommended). `ComplexGitSync` stays where it was cloned, and `bootstrap` clones the whole project tree, root included, into a new workspace of its own, by default under `$HOME/.cgs/CGS<timestamp>/:`

```
$ pixi run cgitsync bootstrap my-project.cgs my-project
$ export CGSHOME=/home/me/.cgs/CGS<timestamp>/my-project # printed by
    bootstrap
$ pixi run cgitsync status
```

Since commands are run from the `ComplexGitSync` clone, the `CGSHOME` variable tells them which workspace to act on. Every command prints the workspace it picked and why — from `$CGSHOME`, or from the current directory — and warns when that is not where you are standing — the usual sign of an `export` left over from an earlier project.

Nested. The project's root repository is cloned first, and `ComplexGitSync` is cloned inside it; `initialise` then builds the rest of the tree around them, with no `export` needed. This suits projects that ship the tool along with their own content, such as hydrological digital twins, where whoever obtains the project obtains the means to synchronise it:

```
$ git clone https://gitlab.com/CGS_test/CGS111.git
$ cd CGS111
$ git clone https://github.com/flipoyo/ComplexGitSync.git
$ cd ComplexGitSync
$ pixi install
$ pixi run cgitsync initialise ../CGS111.cgs
```

Each command refuses the other's job by name before touching the disk: `initialise` run from outside a checked-out root names `bootstrap`, and `bootstrap` into a directory that already holds a checkout names `initialise`. `status` reports which case is in force, as `use_case=standalone` or as `use_case=nested`.

8 A first project

Choose the starting point that matches the project; each has a tutorial that walks through it end to end.

To discover the tool on a sample tree. The small public tree CGSill1 has nothing of your own at risk. `tutorials/01_first_multi_repo_workspace.md` installs it with `bootstrap`, shows the same tree installed nested with `initialise` so the difference is concrete, then runs the whole lifecycle on it.

The project already provides a .cgs. This is the usual case, for any kind of project: use `bootstrap`, as in Section 7. `tutorials/02_onboarding_a_real_build_tree.md` does this for a real 19-repository project.

The project is on disk but has no .cgs. `discover` scans a directory for Git repositories and drafts the `.cgs` from what is checked out. It writes nothing until `-write` is given; review the draft before using it.

```
$ pixi run cgitasync discover ~/work/my-project --write my-project.cgs
$ pixi run cgitasync validate my-project.cgs
```

The project uses git submodules. `submodules init` adopts the checkout in one command — `discover`, write the `.cgs`, `initialise`, then turn each submodule into a plain nested repository — and `-dry-run` shows the plan first. It leaves the conversion staged, not committed. `tutorials/03_adopting_a_real_project.md` walks through it.

```
$ pixi run cgitasync submodules init ~/work/my-project --dry-run
$ pixi run cgitasync submodules init ~/work/my-project
```

9 Day-to-day work

Once the tree is materialised, each Git operation is run once, for the whole tree:

```
$ pixi run cgitasync status                # where every repository
    stands
$ pixi run cgitasync view-tree             # the tree, drawn
$ pixi run cgitasync pull                  # bring every repository up
    to date
$ pixi run cgitasync checkout my-feature   # move the project to a
    branch
... work on the files ...
$ pixi run cgitasync add
$ pixi run cgitasync commit "what changed"
$ pixi run cgitasync push
$ pixi run cgitasync freeze-release v1.0 "first release" # add, commit,
    pull, push, record
```

`status` prints one row per repository — its branch, whether its files are `clean` or `dirty`, and whether it is `synced`, `ahead` or `behind` its remote — and a summary line naming the project's branch. `pull` visits the tree from the root down; `add`, `commit` and `push` from the leaves up, so a parent never records a child that is not yet committed. `add`, `commit`, `push` and `freeze-release` accept `-dry-run` to show what they would do.

Commands that could lose work refuse instead. `merge` checks every repository before merging any, so a conflict anywhere leaves the whole tree untouched; `pull -force` refuses while a commit

exists that no remote holds, and sets uncommitted work aside with `git stash` rather than discarding it.

10 Getting help

```
$ pixi run cgitssync --help           # every command, grouped by
    purpose
$ pixi run cgitssync <command> --help # one command: purpose, options,
    examples
$ pixi run cgitssync help --all       # every command and option on one
    page
```

`-h` is accepted wherever `-help` is. When a command fails, it prints one line saying why, and its full log is kept in `CGSHOME/.cgitssync/logs/`; `cgitssync autofix` reads the last one and diagnoses the situation.

11 Next steps

The five tutorials under `tutorials/` work through the cases above, from the simplest to the most advanced. The User Guide is the reference for every command, option, document format and error.